SIGSPATIAL '26 · APPLICATIONS TRACK

Lexicographic Multi-Preference Routing

Priority-aware urban navigation from open municipal & real-time data

Abdallah Abdelwahab · Ahmad Alsharif · Mahmoud Mahmoud
Department of Computer Science, University of Alabama

github.com/UA-AI-Robustness/lexicographic-freqastar

THE PROBLEM

Navigation gives you a single objective and a few on/off switches.

Every major service folds distance and traffic into one **travel-time** estimate, then exposes a handful of binary toggles:

`avoid tolls` `avoid highways` `avoid ferries`

You *cannot* ask for safer streets, fewer traffic signals, or the roads drivers actually take — and you cannot say **which matters more** when they conflict.

Real people have priorities

A parent at night wants the **safest** route above all.

A courier wants the **fastest** route that still dodges congestion.

A cyclist wants to avoid **signal-heavy** corridors.

OUR INSIGHT

People express what they want as a priority order —
“safety first, then the usual way” —
far more naturally than as a vector of weights.

So we let users **rank** their criteria, and we return one route that **honors that ranking** — never trading a higher-priority preference for a lower one.

TWO OBSTACLES

Why this isn't already solved

01

The criteria aren't on the map

Safety, popularity, congestion, signal delay live in **heterogeneous external data** — crash records, trip histories, live feeds, map tags — each in its own units. They must be put on common ground.

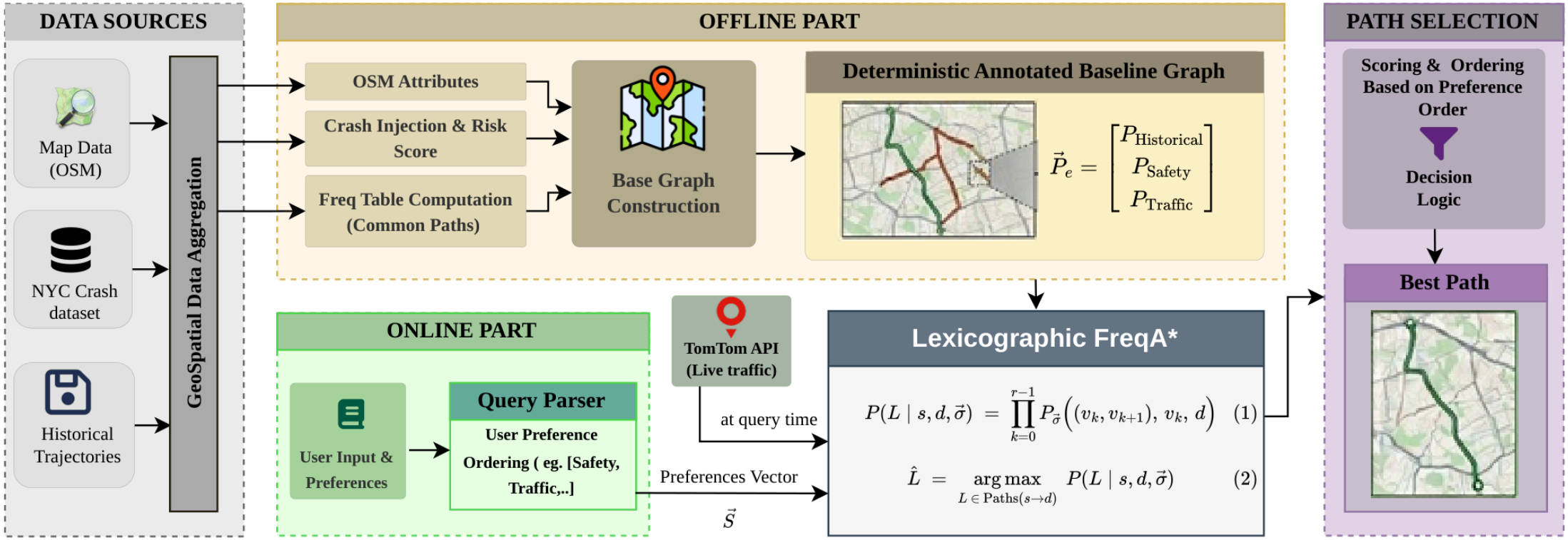
02

Honor an order — fast

Prior work does one of two things: collapse criteria into a **weighted sum** — which can silently override your top preference — or hand you a **Pareto menu** to pick from. Neither honors a stated order, at interactive speed.

SYSTEM OVERVIEW

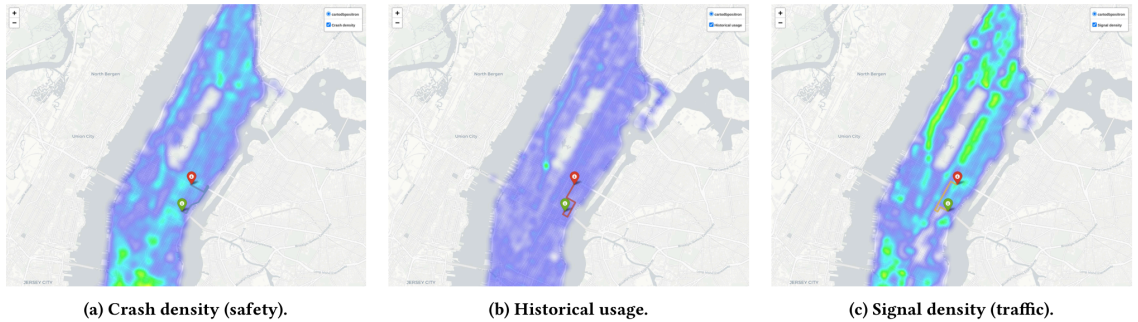
Four open data sources → one priority-ordered route



Offline: convert four urban data sources into an annotated road graph + precompute tile heuristics. Online: the user's priority order (and optional live traffic) feed the search.

PREFERENCES ON A COMMON SCALE

Every source becomes a per-segment score in (0,1]



(a) Crash density (safety). (b) Historical usage. (c) Signal density (traffic).

Crash exposure, historical usage, and signal density over Manhattan — three visibly distinct fields.

One score per preference, per edge. A route's score is the product of its edge scores — a max-product search.

```
safety · crash records
historical · trip popularity
traffic · live speed + signals
```

Because the fields are distinct, different orderings yield genuinely different routes.

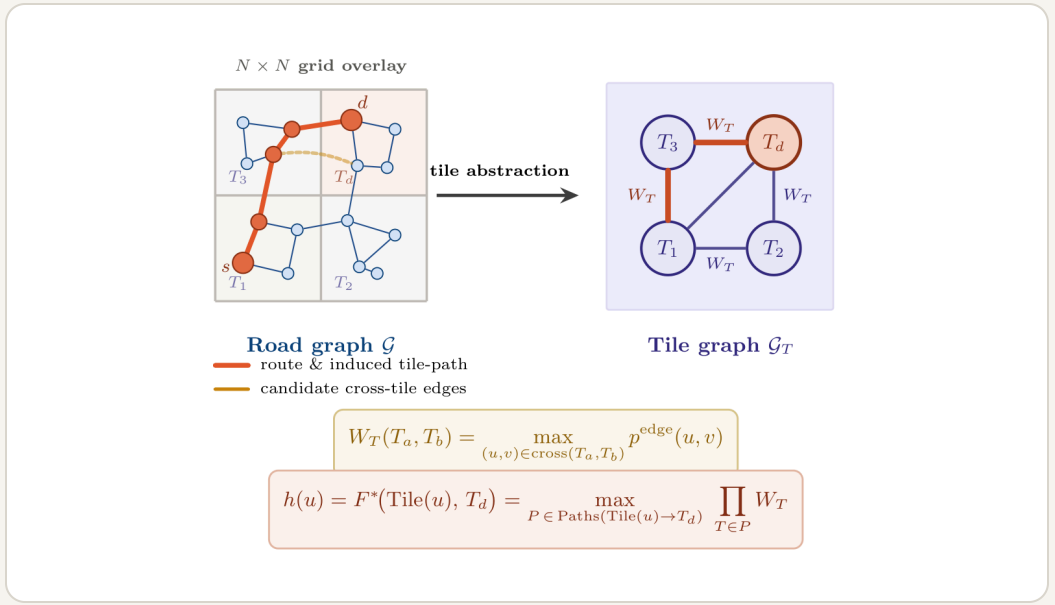
Two ideas: a priority-ordered search, made fast

1 · Lexicographic search

Each route carries a **score vector** — one component per preference. Compare on the top preference first; break near-ties (within ε) by the next. Priorities never multiply, so a lower one can't overrule a higher one.

2 · Tile-graph heuristic

A coarse tile grid gives an **optimistic estimate** of the best score to the destination — an admissible A* heuristic that focuses the search. This is where the 80% node reduction comes from.



The road graph coarsened to a tile graph — heuristic precomputed once per destination, reused for any ordering.

Provably optimal (proofs in paper) — any order served with no offline recomputation.

RESULT 1 · THE ORDER MATTERS

Different priorities → materially different routes

(a) Safety: crashes per route. (b) Historical: popular segments. (c) Offline traffic: signals per route. (d) Online traffic: live congestion.

One O–D pair, four orderings. Each route wins on its own objective; the three offline routes are almost edge-disjoint (pairwise overlap ≤ 2.6%).

RESULT 2 · FAST AND OPTIMAL

Interactive speed, zero quality loss

0.42 ms
per query
(256×256 grid)

80%
fewer node
expansions

100%
FreqScore —
matches optimum

A **97.8× speedup** over the same-objective baseline (FreqDijkstra, 41 ms) — while matching the provably optimal route on **every one** of 197 queries.

METHOD	NODES	TIME
Dijkstra	2348.8	4.07
A*	741.3	2.07
FreqDijkstra	1249.1	41.0
FreqA* (ours)	**249.6**	**0.42**
Lex-FreqA* (ours)	**249.3**	**1.13**

256×256 grid, 197 queries. Time in ms.

RESULT 3 · LIVE TRAFFIC

The same query reroutes as congestion shifts

(a) 07:30, route. (b) 07:30, congestion. (c) 11:42, route. (d) 11:42, congestion.

Traffic-first query at two times (TomTom live feed). Routes keep to free-flowing edges and avoid congested corridors — re-evaluated live, not cached. Per-query search stays at **2.2 ms**.

RESULT 4 · THE COST OF PREFERENCE

Honoring a preference has a price — and a knob

Preference-first routes are longer than the shortest path — a real, interpretable cost, not inefficiency.

ROUTE	VS SHORTEST
Historical-first	+20.3%
Safety-first	+60.1%
Traffic-first	+63.5%

Optional distance budget

Cap the detour at β × shortest path — one feasibility check, no change to the search.

B	DETOUR	SAFETY
∞	+62%	100%
2.0	**+40%**	**85%**
1.25	+13%	52%

β = 2.0 halves the detour, keeps 85% of the safety.

SCOPE & HONESTY

What we claim — and what we don't

- **Synthetic historical corpus.** Popularity is a stand-in; quality is measured against our own field, not real driver choices. Real trajectories drop in without changing the search.
- **Crash exposure, not per-vehicle risk.** Counts aren't traffic-normalized, so the safety detour is likely an upper bound.
- **One city, bounded priority.** Manhattan's grid; strict priority at $\varepsilon = 0$, bounded at the operating tolerance.

Crucially — none of this touches the search guarantees or the efficiency results, which hold for any preference field.

TAKEAWAY

Users should be able to say “safest, then fastest” —

and get a **provably optimal**, priority-respecting route in **under a millisecond**, from **open municipal data**, that **adapts to live traffic**.

0.42 ms
per query

80%
less computation

4
open data sources

`Code & data: github.com/UA-AI-Robustness/lexicographic-freqastar`

Thank you. Questions?

BACKUP · FOR Q&A

Selected related work

Multi-objective & learned routing

Sarker et al. — RL-based multi-objective route recommendation (MASS 2020)

Hung et al. — customizable frequency-guided search (FreqDijkstra)

Liu et al. — driver preferences via inverse RL

Guo et al.; Tian et al. (DRPK) — learned routes from trajectories

Multi-criteria & classical

Martins — multi-objective shortest paths (Pareto)

Kriegel et al.; Shekelyan et al. — route-skyline search

Hu et al.; Wang et al. — crash-risk / congestion-aware routing

Dijkstra; Hart et al. (A*); ALT / CH — classical search

Full citations in the paper. This slide is for Q&A only.